

LeMoonz Portfolio

Technical Documentation & Development Plan

Document type:	Technical project documentation
Planned release:	Version 1.0
Project type:	Responsive portfolio and commissions website
Architecture:	Node.js, Express, EJS, Vanilla JavaScript, CSS
Deployment target:	Railway with custom domain support
Document language:	English
Status:	Active development

Document purpose

This document defines the intended technical scope, architecture, quality standards, deployment model and development roadmap for the LeMoonz portfolio website. It describes both the existing foundation and, more importantly, the target state expected from a complete production-ready release.

This document intentionally contains technical and project information only. It does not include private biographical information, personal background details, or any non-public information about the artist.

Contents

1	Executive Summary	3
2	Project Goals	4
3	Target Audience	5
4	Technical Architecture	6
4.1	Application Model	6
4.2	Architectural Principles	6
5	Planned Project Structure	7
6	Visual Design System	8
6.1	Design Direction	8
6.2	Core Palette	8
6.3	Responsive Behaviour	8
7	Shared Layout Components	9
7.1	Header	9
7.2	Navigation Menu	9
7.3	Footer	9
8	Page Scope	10
8.1	Home	10
8.2	About	10
8.3	Animation	10
8.4	Digital Art	10
8.5	Traditional Art	11
8.6	Other Work	11
8.7	Contact	11
9	Portfolio Data Model	12
10	External Integrations	13
10.1	YouTube Data API	13
10.2	Embedded Third-Party Media	13
11	Privacy and Cookie Strategy	14
12	Error Handling and Reliability	15
12.1	404 Not Found	15
12.2	500 Internal Server Error	15
12.3	External Service Failures	15
13	Accessibility	16
14	Motion and Page Transitions	17
15	SEO and Metadata	18
16	Performance Requirements	19
17	Security	20
17.1	Secrets	20

17.2 HTTP Security	20
17.3 Contact Form Security	20
18 Testing Strategy	21
18.1 Functional Testing	21
18.2 Responsive Testing	21
18.3 Accessibility Testing	21
19 Deployment	22
19.1 Current Hosting Model	22
19.2 Production Deployment Flow	22
20 Development Roadmap	23
20.1 Phase 1 — Foundation	23
20.2 Phase 2 — Portfolio Completion	23
20.3 Phase 3 — Professional Features	23
20.4 Phase 4 — Production Quality	23
20.5 Phase 5 — Release	24
21 Version 1.0 Acceptance Criteria	25
22 Future Development Opportunities	26
23 Maintenance	27
24 Conclusion	28

1 Executive Summary

LeMoonz Portfolio is a custom portfolio website for presenting illustration, animation, traditional artwork and other creative work.

The project uses server-side rendering with Express and EJS, plain CSS and Vanilla JavaScript. The stack stays small on purpose so the code remains easy to understand, maintain and extend.

Version 1.0 is planned as more than a static gallery. It will include structured portfolio pages, selected dynamic data, commission information, contact options, accessibility, SEO, error handling, privacy-aware integrations and a production deployment.

Primary objective

Deliver a fast, accessible, maintainable and visually coherent portfolio website that can serve as the central public-facing platform for the LeMoonz creative brand.

2 Project Goals

The completed platform should:

- present creative work in clearly separated portfolio categories;
- communicate a consistent visual identity across all pages;
- work reliably on desktop, tablet and mobile devices;
- support server-rendered metadata and SEO-friendly routes;
- provide clear navigation with an accessible custom menu;
- include a dedicated contact path for professional enquiries;
- support future commission information and availability states;
- integrate selected external services without exposing private credentials;
- remain usable even when external APIs fail;
- provide custom error pages and graceful fallback behaviour;
- meet a practical accessibility baseline;
- be deployable through Railway and a custom domain;
- remain understandable and maintainable without introducing unnecessary framework complexity.

3 Target Audience

The website is aimed at:

- potential clients looking for commissioned work;
- recruiters, studios and collaborators reviewing a portfolio;
- viewers interested in illustration or animation work;
- existing followers searching for a central source of links and projects;
- visitors arriving from social media, YouTube or other external platforms.

The interface should keep the artistic style while staying easy to navigate, readable and fast.

4 Technical Architecture

4.1 Application Model

The project uses a lightweight server-rendered architecture:

Layer	Technology / responsibility
Runtime	Node.js 24 LTS
Server framework	Express
Template engine	EJS
Frontend structure	Semantic HTML generated from EJS views
Styling	Plain CSS with shared and page-specific stylesheets
Client-side behaviour	Vanilla JavaScript
Static assets	Express static middleware
Dynamic external data	Server-side API requests where appropriate
Environment configuration	Environment variables
Hosting	Railway
Version control	Git and GitHub

4.2 Architectural Principles

Development rules:

- **Progressive complexity:** add infrastructure only when a feature requires it.
- **Server-side rendering first:** core content should remain usable without a large frontend runtime.
- **Separation of concerns:** shared layout, page-specific styling, routing and external service logic should remain distinct.
- **No exposed secrets:** API keys and other credentials must never be delivered to the browser.
- **Graceful degradation:** failure of an external service must not make the site unavailable.
- **Minimal dependencies:** use native platform features where they are sufficient.

5 Planned Project Structure

The codebase should evolve toward a structure similar to the following:

```
project-root/
|
|-- public/
|   |-- css/
|       |-- shared.css
|       |-- home.css
|       |-- about.css
|       |-- animations.css
|       |-- digital.css
|       |-- traditional.css
|       |-- other.css
|       |-- contact.css
|       |-- privacy.css
|       '-- 404.css
|   |
|   |-- js/
|       |-- menu.js
|       '-- page-effects.js
|   |
|   |-- images/
|   |-- videos/
|   |-- favicon.ico
|   |-- robots.txt
|   '-- sitemap.xml
|
|-- src/
|   |-- app.js
|   |-- routes/
|   |-- services/
|       '-- youtubeService.js
|   '-- middleware/
|
|-- views/
|   |-- partials/
|       |-- head.ejs
|       |-- header.ejs
|       '-- footer.ejs
|   |
|   |-- home.ejs
|   |-- about.ejs
|   |-- animation.ejs
|   |-- digital.ejs
|   |-- traditional.ejs
|   |-- other.ejs
|   |-- contact.ejs
|   |-- privacy.ejs
|   |-- 404.ejs
|   '-- 500.ejs
|
|-- .env
|-- .gitignore
|-- package.json
'-- README.md
```

The exact tree may change, but routes, services, views and public assets should stay separated.

6 Visual Design System

6.1 Design Direction

The visual direction is editorial rather than app-like: artistic, mature and slightly book-like.

Core design characteristics include:

- cream paper-like backgrounds;
- dark charcoal typography;
- forest green accents;
- serif typography based on Garamond and Georgia;
- asymmetric image framing and offset decorative rectangles;
- limited use of shadows and rounded corners;
- subtle movement rather than large interface animations;
- generous whitespace and strong typographic hierarchy.

6.2 Core Palette

Name	Value	Intended use
Forest green	#32483A	Accent colour, links, headings, highlights
Cream	#F2EBDD	Primary page background
Dark ink	#1D211E	Main text and dark sections
Soft gray	#4D514D	Secondary text

6.3 Responsive Behaviour

Responsive design must be verified at minimum for:

- mobile screens below approximately 600 px;
- tablet widths;
- common 1080p desktop layouts;
- larger desktop and high-resolution displays.

The implementation should favour `rem`, percentages, `clamp()`, flexbox and CSS Grid where appropriate instead of rigid pixel-based layouts.

7 Shared Layout Components

7.1 Header

The final header should provide:

- logo and public brand name;
- short creative discipline summary;
- persistent access to the navigation trigger;
- sticky behaviour during normal browsing;
- translucent background and backdrop blur;
- responsive simplification on smaller screens.

7.2 Navigation Menu

The menu is a custom full-width overlay and should include:

- numbered navigation items;
- routes for all primary pages;
- hover feedback and directional marker;
- keyboard-accessible menu control;
- `aria-expanded` state updates;
- Escape-key support;
- controlled document scrolling while open;
- a small descriptive footer inside the overlay;
- responsive sizing for mobile devices.

The menu trigger is intentionally allowed to remain visually consistent rather than transforming into a close icon.

7.3 Footer

The footer should provide:

- brand identity;
- creative discipline description;
- copyright information;
- a clear Privacy Policy link;
- optional future links to Terms, commissions or other legal information.

8 Page Scope

8.1 Home

The homepage functions as an overview and conversion surface. Its target content includes:

- introductory artist presentation;
- visual identity and portrait artwork;
- selected inspiration or media content;
- portfolio category previews;
- animation or YouTube-related content;
- contact call-to-action;
- direct routes into deeper portfolio sections.

8.2 About

The About page should expand on professional and creative information without exposing private personal details. Suitable content includes:

- artistic interests and disciplines;
- creative process;
- tools and media;
- professional or educational background where appropriate for public presentation;
- selected process imagery;
- links into portfolio categories.

8.3 Animation

The final Animation page should support:

- featured animation work;
- showreel or selected video presentation;
- project descriptions and roles;
- technical details where useful;
- optional comparison tools such as frame-rate demonstrations;
- links to external video platforms when appropriate.

8.4 Digital Art

The Digital page should provide a curated gallery with:

- optimised thumbnails;
- accessible alternative text;

- optional project titles, dates or categories;
- responsive multi-column layout;
- future support for data-driven rendering from JSON or another content source.

8.5 Traditional Art

The Traditional page should follow a similar structure while preserving presentation appropriate to scanned or photographed physical artwork.

8.6 Other Work

This page is reserved for work that does not fit the main categories. The layout should remain flexible enough to support mixed media, event projects, collaborative work, experiments or selected miscellaneous pieces.

8.7 Contact

The complete Contact page should eventually include:

- clearly presented contact methods;
- links to public profiles;
- optional server-handled contact form;
- form validation;
- spam prevention;
- rate limiting;
- success and failure feedback;
- privacy information explaining how submitted data is processed.

9 Portfolio Data Model

As portfolio content grows, repeated HTML should be reduced by moving project information into structured data.

A future portfolio record may contain fields such as:

```
{
  "title": "Project title",
  "category": "digital",
  "year": 2026,
  "thumbnail": "/images/...",
  "alt": "Accessible artwork description",
  "description": "Short project description",
  "featured": true
}
```

EJS templates can then generate reusable gallery items from a data collection.

This approach improves:

- consistency;
- maintainability;
- future filtering;
- easier content updates;
- reduced duplication.

A database is not required at the current scale. Local structured data is sufficient until content management requirements justify additional infrastructure.

10 External Integrations

10.1 YouTube Data API

The website may use the YouTube Data API to retrieve public channel statistics.

The production implementation should:

- perform requests server-side;
- keep the API key in environment variables;
- validate `response.ok`;
- verify returned data before reading nested fields;
- catch request and parsing errors;
- return a safe fallback when the API is unavailable;
- cache results for a reasonable period to avoid unnecessary API requests.

The homepage must remain functional even when the YouTube integration is unavailable.

10.2 Embedded Third-Party Media

Third-party embeds may include services such as Spotify or YouTube.

Before production release, every embed should be reviewed for:

- privacy implications;
- cookies or tracking technologies;
- page-load impact;
- accessibility;
- lazy-loading possibilities;
- whether a click-to-load placeholder would be preferable.

11 Privacy and Cookie Strategy

A Privacy Policy is part of the website structure.

The final privacy implementation should accurately reflect the actual production behaviour of the site. The policy must be updated whenever new data-processing features are added.

A cookie banner should **not** be added automatically simply because many websites use one. The production decision should be based on the technologies actually loaded.

Potential cases requiring review include:

- analytics;
- advertising;
- tracking scripts;
- embedded media that sets non-essential cookies;
- contact forms that store or transmit personal data.

Where practical, privacy-conscious alternatives should be preferred, such as server-side data retrieval, minimal analytics, delayed third-party embeds or click-to-load content.

12 Error Handling and Reliability

12.1 404 Not Found

The project includes a custom 404 experience. In production, the final catch-all middleware must be placed after all normal routes and must return an actual HTTP 404 status.

12.2 500 Internal Server Error

A production version should also include a custom 500 page and central Express error middleware.

The intended behaviour is:

1. log the technical error server-side;
2. avoid exposing stack traces to visitors;
3. return HTTP status 500;
4. render a branded, understandable error page;
5. provide a route back to the main website.

12.3 External Service Failures

No external API or embed should be a single point of failure for the main website.

For example:

Condition	Required result
YouTube API available	Display current public statistic
YouTube API unavailable	Hide statistic or display safe fallback
Missing optional image	Preserve layout and provide fallback
Invalid route	Render custom 404 page
Unexpected server error	Render custom 500 page

13 Accessibility

Accessibility is part of the release requirements.

The target baseline includes:

- semantic HTML landmarks;
- one clear primary heading per page;
- logical heading hierarchy;
- descriptive alternative text for meaningful images;
- empty alt text for purely decorative images where appropriate;
- keyboard-accessible navigation;
- visible focus states;
- sufficient colour contrast;
- meaningful link text;
- appropriate ARIA attributes only where native HTML is insufficient;
- support for reduced motion preferences;
- no critical interaction requiring hover only.

A final keyboard-only review should be completed before version 1.0.

14 Motion and Page Transitions

Motion should stay subtle and should not get in the way of navigation or reading.

Recommended effects include:

- subtle opacity transitions;
- small vertical or horizontal movement;
- restrained hover movement on artwork;
- lightweight menu transitions;
- optional content reveal on scroll.

Avoid:

- long blocking page transitions;
- excessive parallax;
- animation required to understand content;
- large JavaScript animation libraries without a clear need.

If page-entry effects are added, the site should respect:

```
@media (prefers-reduced-motion: reduce) {  
  /* Disable or significantly reduce non-essential motion. */  
}
```

15 SEO and Metadata

The final release should include:

- unique page titles;
- unique meta descriptions;
- canonical URLs;
- Open Graph metadata;
- social preview image;
- favicon;
- `robots.txt`;
- `sitemap.xml`;
- meaningful route names;
- correct heading structure;
- descriptive image alternative text;
- custom-domain URLs in metadata.

SEO configuration should be finalised only after the permanent production domain and final route structure are confirmed.

16 Performance Requirements

Media optimisation is one of the main performance concerns because the site is image-heavy.

The final implementation should aim to:

- serve appropriately sized images;
- use modern formats such as WebP or AVIF where practical;
- lazy-load non-critical media;
- avoid serving extremely high-resolution originals as small thumbnails;
- compress video assets;
- minimise render-blocking resources;
- keep JavaScript small and feature-specific;
- cache suitable static assets;
- avoid repeated unnecessary third-party requests.

Target performance should be evaluated with browser developer tools and Lighthouse before production launch.

17 Security

The production release should include basic security measures.

17.1 Secrets

- API keys must remain in environment variables.
- `.env` must remain excluded from Git.
- no credential may be exposed in EJS output or browser-side JavaScript.

17.2 HTTP Security

A production setup may use middleware such as Helmet to configure common HTTP security headers.

17.3 Contact Form Security

If a server-side contact form is added, it should include:

- server-side validation;
- input length limits;
- sanitisation appropriate to the output context;
- rate limiting;
- spam protection;
- no direct trust of browser-provided fields;
- generic error responses without leaking server details.

18 Testing Strategy

18.1 Functional Testing

Before release, verify:

- all routes;
- menu open and close behaviour;
- Escape-key menu handling;
- all internal links;
- all external links;
- custom 404 handling;
- error fallbacks;
- dynamic API content;
- contact workflow once implemented.

18.2 Responsive Testing

Test on:

- modern Firefox;
- Chromium-based browser;
- Android or iOS mobile browser;
- small mobile viewport;
- tablet viewport;
- 1080p desktop;
- larger desktop display.

18.3 Accessibility Testing

Perform:

- keyboard-only navigation;
- visible-focus inspection;
- browser accessibility-tree inspection where necessary;
- basic automated accessibility audit;
- manual review of image alternative text.

19 Deployment

19.1 Current Hosting Model

The application is designed for deployment to Railway.

Railway provides:

- Node.js application hosting;
- environment variables;
- automatic deployment from GitHub;
- generated production URL;
- support for custom domains and HTTPS.

19.2 Production Deployment Flow

The intended workflow is:

1. develop and test locally;
2. commit changes to Git;
3. push to the GitHub repository;
4. allow Railway to build and deploy automatically;
5. verify production logs;
6. perform a smoke test on the public deployment;
7. attach the final custom domain;
8. update canonical URLs, sitemap and social metadata;
9. tag the stable release as version 1.0.

20 Development Roadmap

20.1 Phase 1 — Foundation

Mostly implemented

- Express application setup;
- EJS rendering;
- shared static assets;
- core visual language;
- reusable head, header and footer partials;
- responsive homepage foundation;
- custom navigation system;
- Railway deployment;
- environment-based API configuration;
- Privacy Policy;
- custom 404 handling.

20.2 Phase 2 — Portfolio Completion

Primary next development phase

- complete About page;
- complete Animation page;
- complete Digital Art gallery;
- complete Traditional Art gallery;
- complete Other Work page;
- add structured reusable portfolio data;
- optimise gallery media;
- review all routes and calls-to-action.

20.3 Phase 3 — Professional Features

- build a complete Contact page;
- optionally add a secure contact form;
- add commissions information and availability state;
- add terms relevant to commission work if required;
- improve dynamic YouTube integration;
- add robust fallback handling.

20.4 Phase 4 — Production Quality

- custom 500 page;
- accessibility audit;

- performance optimisation;
- responsive cross-browser testing;
- image conversion and resizing;
- final metadata;
- Open Graph;
- sitemap;
- robots configuration;
- HTTP security headers;
- privacy review for third-party content.

20.5 Phase 5 — Release

- connect permanent custom domain;
- verify HTTPS;
- production smoke test;
- final content review;
- final legal/privacy review;
- release version 1.0;
- create maintenance and backup routine.

21 Version 1.0 Acceptance Criteria

Version 1.0 should be considered production-ready only when all of the following are true:

1. all intended public pages exist and contain production content;
2. there are no broken internal links;
3. the site is usable on mobile and desktop;
4. primary navigation works with mouse and keyboard;
5. privacy information matches the actual technical implementation;
6. no secret is present in frontend source or Git history intended for publication;
7. external API failure does not break the homepage;
8. custom 404 and 500 responses are present;
9. large media assets have been optimised;
10. metadata is configured for the final domain;
11. Lighthouse and manual checks reveal no major accessibility or performance issue;
12. deployment is stable on Railway;
13. the permanent domain and HTTPS operate correctly.

22 Future Development Opportunities

Features that may be considered after version 1.0 include:

- lightweight content management interface;
- project filtering and tagging;
- featured-work management;
- automated thumbnail generation;
- image lightbox or project detail views;
- case-study pages for larger animation projects;
- commission availability control;
- privacy-friendly analytics;
- multilingual content;
- automated deployment checks;
- basic unit or integration tests for backend services.

These features are intentionally outside the core release scope unless a concrete business or content-management need appears.

23 Maintenance

After launch, routine maintenance should include:

- dependency updates;
- periodic production log review;
- broken-link checks;
- API integration checks;
- image and content updates;
- privacy-policy updates after technical changes;
- accessibility checks after significant interface changes;
- domain and hosting renewal monitoring;
- backup of repository and important media assets.

24 Conclusion

LeMoonz Portfolio is a custom website rather than a generic template. The project keeps full control over layout and behaviour while avoiding unnecessary dependencies.

The base of the site is already in place. Remaining work is mostly portfolio content, production hardening, accessibility, performance, contact features and final deployment.

After the roadmap is completed, the site will be ready to work as the main public portfolio and contact point for LeMoonz.